

# Demystifying MNIST: The Math and Intuition of Digit Classification

## A Theoretical Guide for Beginners

### Abstract

This document serves as a theoretical foundation for implementing a handwritten digit classifier using the MNIST dataset. It deliberately omits code implementation to focus entirely on the conceptual architecture, mathematical underpinnings, and intuition of a fundamental Neural Network (Multilayer Perceptron).

## 1 1. The Problem Space: Seeing as a Computer

The MNIST dataset consists of grayscale images of handwritten digits (0-9). To a human, an image is a cohesive shape. To a computer, an image is merely a grid of numbers representing pixel intensities.

Each MNIST image is  $28 \times 28$  pixels. Before we can feed this into a standard neural network, we must "flatten" this 2D grid into a 1D array (a vector).

$$28 \times 28 = 784 \text{ pixels}$$

Thus, the input to our program is a single vector  $X$  with 784 dimensions, where each value ranges from 0 (black) to 255 (white)—typically normalized to be between 0.0 and 1.0.

## 2 2. Architecture: The Multilayer Perceptron (MLP)

Our network acts as a function that maps the 784-dimensional input vector to a 10-dimensional output vector (representing the 10 possible digits). We structure this using layers:

- **Input Layer:** 784 nodes (our flattened image).
- **Hidden Layer(s):** Intermediate nodes where the network learns internal representations (e.g., detecting edges, loops, or curves). Let's assume a single hidden layer of 128 nodes for simplicity.
- **Output Layer:** 10 nodes (one for each digit).

Connections between these layers are defined by **Weights** ( $W$ ) and **Biases** ( $b$ ). The network's entire memory and knowledge are stored in these parameters.

## 3 3. Forward Propagation: Making a Guess

Forward propagation is the process of passing the input data through the network to get a prediction. It happens in two steps per layer: a linear transformation followed by a non-linear activation.

### 3.1 The Linear Step

For our hidden layer, we calculate the weighted sum of the inputs and add a bias. If  $X$  is our input vector,  $W^{[1]}$  is the weight matrix connecting the input to the hidden layer, and  $b^{[1]}$  is the bias vector:

$$Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$$

*Intuition:* The weights act as "filters" looking for specific patterns in the pixels. The bias shifts the result, acting as a threshold for activation.

### 3.2 The Non-Linear Step (Activation)

If we only use linear transformations, stacking layers would be mathematically pointless (a linear combination of linear combinations is just another linear combination). We introduce non-linearity using an **Activation Function**. A common choice is **ReLU** (Rectified Linear Unit):

$$A^{[1]} = \max(0, Z^{[1]})$$

*Intuition:* ReLU mimics a biological neuron firing. If the weighted sum is negative, the neuron doesn't fire (outputs 0). If positive, it fires proportional to the input.

## 4 4. The Output: Probabilities via Softmax

After the hidden layer, we pass the data to the final output layer:

$$Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$$

$Z^{[2]}$  gives us 10 raw scores (logits). However, raw scores are hard to interpret. We want probabilities that sum to 100%. We achieve this using the **Softmax** function:

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=0}^9 e^{z_j}}$$

*Intuition:* The exponential function ensures all values are positive and exaggerates differences (making the most likely digit stand out). Dividing by the sum normalizes the values so they add up to 1.0. If  $\hat{y}_3 = 0.85$ , the network is 85% confident the image is a '3'.

## 5 5. Measuring Error: Cross-Entropy Loss

To learn, the network must know how wrong it is. We compare our predicted probabilities ( $\hat{y}$ ) against the true label ( $y$ ). The true label is one-hot encoded (e.g., for a '3',  $y = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$ ).

We measure the error using **Categorical Cross-Entropy**:

$$L = - \sum_{i=0}^9 y_i \log(\hat{y}_i)$$

*Intuition:* Because  $y_i$  is 0 for all incorrect classes, this formula simplifies to  $-\log(\hat{y}_{\text{correct\_class}})$ . If the network predicted 1.0 for the correct class,  $-\log(1) = 0$  (no error). As the predicted probability drops towards 0, the penalty (loss) grows exponentially.

## 6 6. Learning: Gradient Descent and Backpropagation

With the error calculated, the network must update its weights and biases to reduce this error.

### 6.1 Gradient Descent

Imagine standing on a foggy mountain (the loss landscape) trying to reach the bottom. You can't see the bottom, but you can feel the slope under your feet. By taking a step downhill, you get closer to the minimum. Mathematically, the "slope" is the **Gradient** (the derivative of the Loss with respect to the weights).

$$W_{\text{new}} = W_{\text{old}} - \alpha \frac{\partial L}{\partial W}$$

Here,  $\alpha$  is the **Learning Rate**—the size of the step you take.

### 6.2 Backpropagation

To find these gradients ( $\frac{\partial L}{\partial W}$ ), we use Backpropagation, which is simply the application of the **Chain Rule** from calculus. We calculate how much a small change in the output affects the loss, then how much a change in the hidden layer affects the output, working backward to the input.

$$\frac{\partial L}{\partial W^{[2]}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

*Intuition:* We are assigning "blame" for the error. If a specific weight caused the network to confidently predict '8' when the image was a '3', backpropagation calculates exactly how to adjust that weight down so it makes a better prediction next time.

## 7 7. The Overall Architecture

Your implementation will repeatedly loop through these steps for a set number of *epochs*:

1. **Sample:** Take a batch of images and their true labels.
2. **Forward Pass:** Calculate linear combinations and activations to get predictions.
3. **Loss:** Calculate how far off the predictions are.
4. **Backward Pass:** Use the chain rule to calculate gradients.

5. **Update:** Nudge the weights and biases in the opposite direction of the gradient.

Over time, the weights will organize themselves to recognize the loops, lines, and curves that distinguish the ten digits.